

CSC BIOINFORMATICS PROJECT

High-Performance Port of Scanpy to Codon

Accelerating Single-Cell RNA Analysis Through Compiled Kernels

The Problem

Why Python struggles with single-cell analysis

Data Scale

1M+

cells per experiment

20-30K

genes measured

Python Bottlenecks

Global Interpreter Lock

Prevents true parallelism

Dynamic Type Checking

Runtime overhead in loops

Interpreter Overhead

Per-operation dispatch cost

Solution: Codon

LLVM Compilation

Native machine code

Static Type Inference

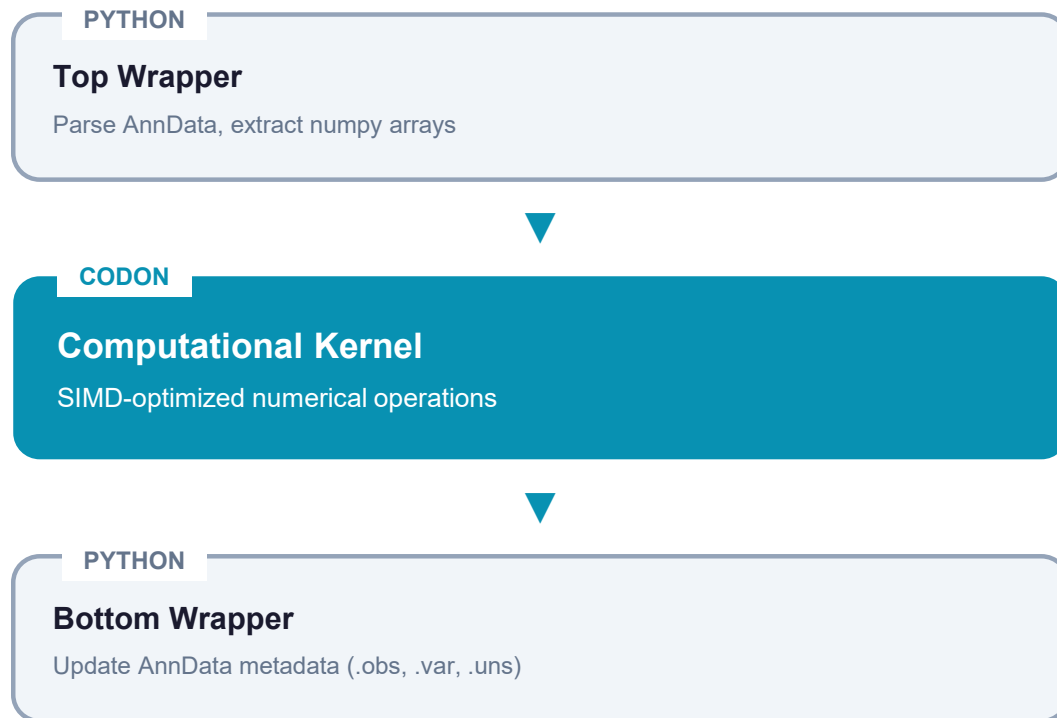
No runtime checking

SIMD Vectorization

Hardware acceleration

The "Sandwich" Architecture

Hybrid Python/Codon design pattern



Why This Works

Python handles complexity

AnnData objects, API design, sparse conversion

Codon handles speed

Tight loops, matrix operations, statistics

API compatible

Drop-in replacement with fallbacks

Challenge: Linking

Custom gcc script to resolve runtime symbols

Implementation Details

Key computational kernels



Differential Expression

rank_genes_groups

Python Approach:

Loop over 20,000+ genes, call scipy.stats for each

Codon Approach:

Single matrix-wide operation with vectorized Welch's t-test

48x

speedup



k-NN Computation

pp.neighbors

Matrix Multiplication Identity:

$$||x_i - x_j||^2 = ||x_i||^2 + ||x_j||^2 - 2(x_i \cdot x_j)$$

Approach:

Exact Euclidean via optimized BLAS routines

Complexity

$O(N^2)$

Status

Parity

The "Smart Cache" Fix

Solving the Bridge Tax problem

Problem: UMAP wrapper was SLOWER than Python

Each Python-Codon transition incurs marshalling overhead ("Bridge Tax"). UMAP was re-densifying matrices and recomputing neighbour graphs unnecessarily.

BEFORE

pp.neighbors

▼ discard

tl.umap recomputes

▼ slow

+2.8s overhead

CACHE in adata.uns

`_codon_dense_X`
Densified expression matrix

`_codon_knn_indices`
Neighbour indices

`_codon_knn_distances`
Corresponding distances

AFTER

pp.neighbors

▼ cache

tl.umap detects cache

▼ inject

3.6s to 1.1s

Validation & Testing

The "Hijack" methodology for correctness verification

Testing Approach

Hijack Scanpy's Tests

Reuse existing test patterns with identical inputs, parameters, and expected outputs from the mature Scanpy library.

Numerical Tolerance

Compare outputs using `numpy.allclose` with relative tolerance 10^{-5} and absolute tolerance 10^{-8} for floating-point results.

Custom Harness

`evaluate_codon.py` runs without `pytest` (incompatible with `Codon`), executing 34 validation checks across 5 test suites.

Test Coverage

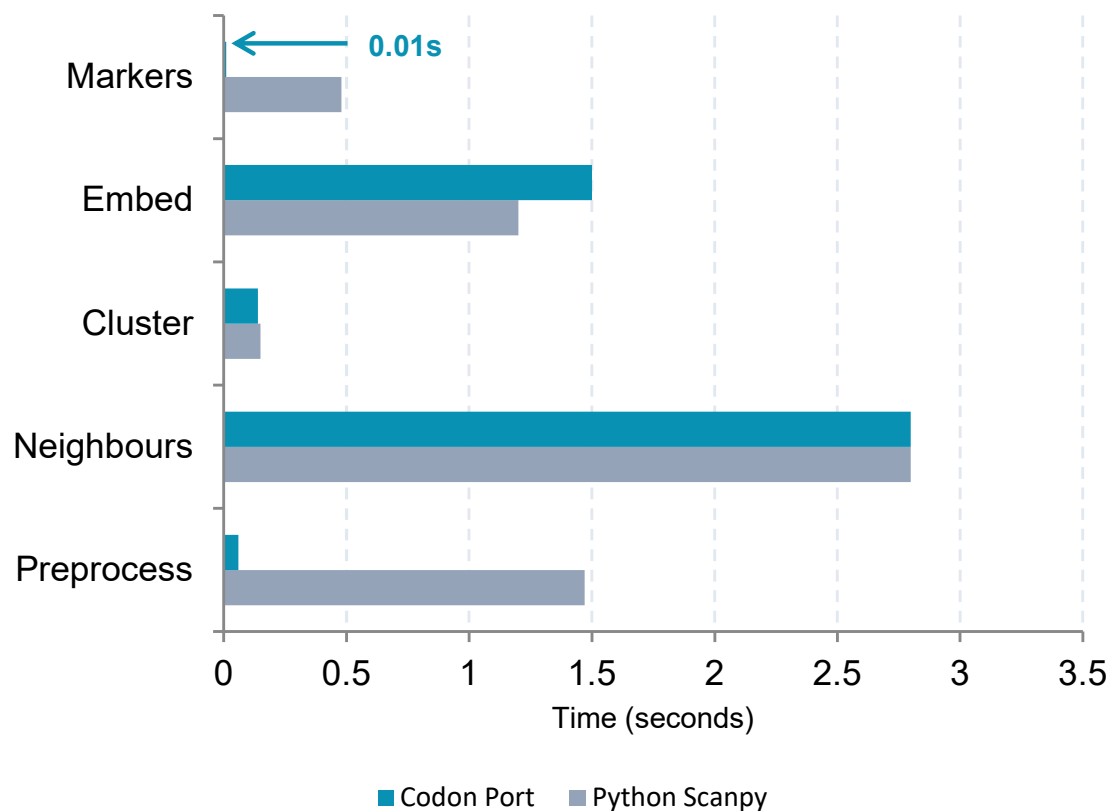
Preprocessing	normalize, log1p, scale, filter
Neighbours	k-NN graph construction
Clustering	Leiden community detection
Embedding	UMAP dimensionality reduction
Markers	t-test, Wilcoxon rank-sum

34/34

tests passing in ~2.6s

Benchmark Results

PBMC 3k dataset (2,700 cells, ~13,000 genes)



53%

Total Pipeline Reduction
5.60s to 2.60s

Top Speedups

Preprocessing	24.5x
Markers (t-test)	48x
Neighbours	1.0x (parity)

Validation

34/34

tests passing in ~2.6s

Performance at a Glance

24.5x

Preprocessing

1.47s to 0.06s

48x

Marker Genes

0.48s to 0.01s

2.2x

Total Pipeline

5.60s to 2.60s

1.0x

Neighbours

Parity achieved

1.1x

Clustering

Leiden via igraph

0.8x

Embedding

+0.3s bridge overhead

Limitations & Future Work

Current Limitations

Memory Complexity

$O(N^2)$ distance matrix limits scalability to ~50K cells

Sparse Handling

Forced densification wastes memory for large sparse matrices

Parameter Coverage

Some advanced features fall back to Python Scanpy

Future Improvements

Sparse CSR Kernels

Native iteration over non-zero elements for million-cell datasets

Approximate k-NN

Annoy/HNSW for $O(N \log N)$ neighbour search

Thread Parallelism

OpenMP-style `@par` annotations for multi-core

Extended Coverage

Port `draw_graph`, `dpt`, `paga`, `diffmap`

Key Takeaways

1

Codon as Accelerator, Not Replacement

Python handles complex object models (AnnData). Codon handles tight numerical loops. Each layer operates in its optimal domain.

2

Minimize Bridge Crossings

The "Smart Cache" pattern eliminates redundant computation. Cache intermediate results to avoid repeated Python-Codon transitions.

3

Manual Linking Resolves Dependencies

Standard build tools may fail. Custom gcc scripts ensure both libpython and libcodonrt symbols are resolved.

This hybrid approach generalizes to any Python library with performance-critical numerical loops.